koroteevmv / ML_course
Type / to search
Code
Issues 2
Pull requests
Agents
Actions
Projects
Security and quality
Insights
Files
main
Go to file
ML4.1 metrics
ML4.2 threshold
ML4.3 cross-validation
ML4.4 diagnostics
ML4.5 hyperparameters
ML5.1 data integration
ML5.2 numeric features
ML5.3 categorical features
ML5.4 target
ML5.5 EDA
ML6.1 clustering
ML6.2 pca
ML7.1 text
ML7.4 images
misc
LICENSE
README.md
README_template.md
ML_course / ML5.5 EDA /
Add file
koroteev-test Картинки распределены по папкам
7dd1f73 · 2 years ago
History
README.md

Name	Last commit message	Last commit date
..		
data	Картинки распределены по папкам	2 years ago
img	Черновик методички ML5.2	2 years ago
README.md	Картинки распределены по папкам	2 years ago

Предобработка данных

Цель работы

Познакомиться с основными задачами и приемами предварительного анализа и обработки данных для целей машинного обучения

Предварительная обработка данных является неотъемлемым этапом машинного обучения, поскольку качество данных и полезная информация, которую можно извлечь из них, напрямую влияют на способность нашей модели к обучению; поэтому чрезвычайно важно, чтобы мы предварительно обработали наши данные, прежде чем вводить их в нашу модель.

Содержание работы

1. Загрузите данные о пассажирах Титаника и познакомьтесь со структурой датасета.
2. Проведите анализ и визуализацию каждого признака датасета. Сделайте вывод о виде распределения и шкале каждого признака.
3. Проанализируйте влияние каждого признака на целевую переменную. Проиллюстрируйте ее графиками.
4. Исследуйте и исправьте при необходимости пропущенные значения в датасете.
5. Преобразуйте категориальные признаки в численные самым подходящим способом.

Методические указания

Подготовка и загрузка данных

Для полноценной работы с регрессионным анализом данных нам потребуются следующие библиотеки языка Python:

```
import pandas as pd
import numpy as np
import seaborn as sns
```

Если вы работаете в ноутбуке Jupyter или Google Colab, то для лучшего отображения графиков следует выполнить следующие инструкции:

```
import matplotlib.pyplot as plt
%matplotlib inline
```

Для начала считаем данные из csv-файла:

```
training_set = pd.read_csv('https://raw.githubusercontent.com/koroteevmv/ML_course/2023/ML5.5%20EDA/data/titanic.csv')
```

Метод .head() печатает первые 5 строк из обучающей выборки

```
training_set.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Результат работы функции - предпросмотр загруженных данных. Рассмотрим внимательнее, что содержится в данном датасете. Ниже представлена краткая информация о каждом из столбцов датасета:

PassengerId: Уникальный индекс/номер строки. Начинается с 1 (для первой строки) и увеличивается на 1 для каждой следующей. Рассматриваем его как идентификатор строки и, что логично, идентификатор пассажира (т.к. для каждого пассажира в датасете представлена только одна строка).

Survived: Признак, показывающий был ли спасен данный пассажир или нет. 1 означает, что удалось выжить, и 0 - не удалось спастись.

Pclass: Класс билета. 1 - означает Первый класс билета. 2 - означает Второй класс билета. 3 - означает Третий класс билета.

Name: Имя пассажира. Имя также может содержать титулы и обращения. "Mr" для мужчин. "Mrs" для женщин. "Miss" для девушек (тут имеется в виду что для тех, кто не замужем, так было принято, да и сейчас тоже, говорить в западном обществе). "Master" для юношей.

Sex: Пол пассажира. Либо мужчины (=Male) либо женщины (=Female).

Age: Возраст пассажира. "NaN" значения в этой колонке означают, что возраст данного пассажира отсутствует/неизвестен/или не был записан в датасет.

SibSp: Количество братьев/сестер или супругов, путешествующих с каждым пассажиром.

Parch: Количество родителей детей (Number of parents of children travelling with each passenger).

Ticket: Номер билета.

Fare: Сумма, которую заплатил пассажир за путешествие.

Cabin: Номер каюты пассажира. "NaN" значения в этой колонке указывает на то, что номер каюты данного пассажира не был записан.

Embarked: Порт отправления данного пассажира.

.describe() отобразит различные величины, такие как количество, среднее, среднеквадратичное отклонение и т.д. для численных типов данных.

Это может быть полезным для понимания распределения значений по датасету и статистики, особенно когда нет возможности просмотреть все записи в виду огромного их количества

```
training_set.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Очень часто во множестве данных будут присутствовать отсутствующие данные. Используем метод isnull. Результатом вызова данного метода является логическое значение, указывающее, действительно ли значение, переданное в аргумент, отсутствует. «Истина» (True) означает, что значение является отсутствующим значением, а «Ложь» (False) означает, что значение не является отсутствующим.

.describe(include = ['O']) отобразит статистики (descriptive statistics) объектного типа. Это нужно для нечисловых данных, когда нельзя просто посчитать максимумы/среднее/и пр. для данных. Мы можем отнести такие данные к категориальному виду

```
training_set.describe(include=['O'])
```

	Name	Sex	Ticket	Cabin	Embarked
count	891	891	891	204	889
unique	891	2	681	147	3
top	Braund, Mr. Owen Harris	male	347082	B96 B98	S
freq	1	577	7	4	644

Видно, что существуют дубликаты номеров билетов Ticket и переиспользуются каюты Cabins (уникальных записей (unique) меньше, чем общего количества). Самый большой порядок дубликата билета - "347082". Он повторился 7 раз. Аналогично, наибольшее число людей, занимающих одну и ту же каюту - 4. Они используют каюты "B96 B98". Также можно заметить, что 644 человека отбыли из порта "S". Среди 891 записей, 577 были мужчины (Male), оставшиеся - женщины (Female).

Описание каждого признака

Гистограмма распределения признаков

```
def custom_hist(training_set, title,  xlabel, ylabel='Количество', bins=None):
    figsize = (20,6)
    plt.figure(figsize=figsize)
    plt.grid(True)
    plt.title(title)
    plt.hist(training_set, training_set.max().astype(int) if bins is None else bins)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.show()
```

```
custom_hist(training_set["Age"], 'Распределения пассажиров по возрасту', 'Возраст')
```

```
custom_hist(training_set["SibSp"], 'Распределения пассажиров по количеству братьев/сестер или супругов',
  'Число братьев/сестер или супругов')
```

```
custom_hist(training_set["Parch"], 'Распределения пассажиров по количеству родителей или детей',
  'Число родителей или детей')
```

```
custom_hist(training_set["Fare"], 'Распределения пассажиров по стоимости билета',
  'Стоимость билета', bins=20)
```

Далее проанализируем возраст людей, это мы сделаем с помощью графика распределения. На графике мы видим, что средний возраст пассажиров составляет 20-35 лет.

```
training_set['Age'].plot.hist(bins=30)
```

Проведем исследования столбца SibSp, означающий сестра,братья/супруги. Из графика мы видим, что большинство не имело братьев и сестер, а так же супругов, следующий столбец -1- учитывает супругов

```
sns.countplot(x='SibSp', data=training_set)
```

Следующий столбец который будем исследовать - Fare, обозначает сколько люди платили за билет.

```
training_set['Fare']
```

```
0       7.2500
1      71.2833
2       7.9250
3      53.1000
4       8.0500
        ...   
886    13.0000
887    30.0000
888    23.4500
889    30.0000
890     7.7500
Name: Fare, Length: 891, dtype: float64
```

Построим гистограмму:

```
training_set['Fare'].hist()
```

Из этого графика мы видим, что наибольшее распределение от 0 до 150, рассмотрим этот диапазон более подробно. Из графика мы видим, что наибольшее количество билетов были по цене до 50. Это действительно так, так как мы видим из данных, что большинство пассажиров было из третьего класса.

```
training_set['Fare'].hist(bins=40, figsize=(10,4))
```

Описание вида совместного распределения

Установим соотношение выживших и не выживших

```
sns.countplot(x='Survived', data=training_set)
```

Отобразим пол выживших и не выживших. На графике мы видим, что среди не выживших большинство было мужчин, в выживших наоборот больше было женщин

```
sns.countplot(x='Survived', data=training_set, hue='Sex')
```

В параметре hue мы можем использовать другой столбец, например класс пассажира Pclass, и посмотреть как это соотносится с количеством выживших. Проанализировав полученный график, можно сказать, что из не выживших было больше людей третьего класса

```
sns.countplot(x='Survived', data=training_set, hue='Pclass')
```

Создадим boxplot, передадим три параметра, класс пассажира, возраст, датасет. Из полученного графика можно сделать вывод, что средний возраст пассажиров первого класса больше чем средний возраст пассажиров второго класса, и соответственно средний возраст пассажиров второго класса больше чем средний возраст пассажиров третьего класса. Мы можем использовать эти средний значения для того что бы вставлять эти значения там где они отсутствуют, основываясь на классе.

```
sns.boxplot(x='Pclass', y='Age', data=training_set)
```

В процессе анализа данных мы попробуем увидеть зависимости целевого признака от остальных признаков и остальных признаков между собой, чтобы избежать мультиколлинеарности и выбрать признаки, которые не имеют значения и которые стоит удалить.

Для признаков class, sex, sib_sp, par_ch, embarked визуализируем доли выживших для каждого значения признака:

```
columns_to_look = ['Pclass', 'Sex', 'SibSp', 'Parch', 'Embarked']

for column in columns_to_look:
    pivot = training_set.pivot_table(index=column, values='Survived', aggfunc='mean')

    fig, ax = plt.subplots(figsize=(15,5))
    ax.set_title(f'Доля выживших по признаку {column}', fontdict={'size': 16})
    ax.set_ylabel('Доля выживших', fontdict={'size': 12})
    ax.set_xlabel(column, fontdict={'size': 12})

    for cnt in range(pivot.shape[0]):
        value = pivot.iloc[cnt].values[0]
        ax.text(cnt - .05, value + .005, round(value, 2))

    pivot.plot(kind='bar', rot=0, grid=True, legend=False, ax=ax)
    ax.set_xlabel(f'Значения признака {column}', fontdict={'size': 12})
    plt.show()
```

Доля выживших по признаку Pclass
0.63
0.47
0.24
Доля выживших по признаку Sex
0.74
0.19
Доля выживших по признаку SibSp
0.35
0.54
0.46
0.25
0.17
0.0
0.0
Доля выживших по признаку Parch
0.34
0.55
0.5
0.6
0.0
0.2
0.0
Доля выживших по признаку Embarked
0.55
0.39
0.34

Видим, что ни один из признаков не стоит убирать из таблицы, т.к. значения доли выживших сильно отличается. Разумеется, различия могуть быть вызваны случайностью, особенно на больших значениях признаков SibSp и Parch, т.к. объектов с такими значениями мало и доля выживших не очень информативна. Но удалить эти признаки мы всё-таки не можем, т.к. значения доли различны.

Видно, что особенно значимое влияние на значение целевого признака оказывает пол пассажира. Довольно значимым признаком так же является класс.

Исследование пропущенных значений

Используем метод .info(), чтобы увидеть больше информацию о типах данных/структуре в тренировочной выборке.

```
training_set.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Можно увидеть, что значение Age не задано для большого количества записей. Из 891 строк, возраст Age задан лишь для 714 записей. Аналогично, номер каюты "Cabin" также пропущены для большого количества записей. Только 204 из 891 записей содержат значения Cabin.

```
training_set.isnull().sum()
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

Всего 177 записей с пропущенным возрастом (Age), 687 записей с пропущенным значение каюты Cabin и для 2 записей не заданы порты отправления Embarked.

Исследование отсутствующих значений

Приступим к исследованию отсутствующих значений.

При вызове метода isnull получаем таблицу с булевыми значениями, False - присутствуют данные, True - данные отсутствуют

```
training_set.isnull()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	False	False	False	False	False	False	False	False	False	False	True	False
1	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	True	False
3	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	True	False
...	...	...	...	...	...	...	...	...	...	...	...	...
886	False	False	False	False	False	False	False	False	False	False	True	False
887	False	False	False	False	False	False	False	False	False	False	False	False
888	False	False	False	False	False	True	False	False	False	False	True	False
889	False	False	False	False	False	False	False	False	False	False	False	False
890	False	False	False	False	False	False	False	False	False	False	True	False

Еще раз посмотрим что находится в training_set

```
training_set.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Для визуализации представленых булевых значений воспользуемся Seaborn heatmap, с помощью чего сможем увидеть где больше всего отсутствующих данных. Желтый цвет нам говорит о пропущенных значениях. Наглядно видно, что много отсутствующих данных в столбце Age и Cabin.

```
sns.heatmap(training_set.isnull(), yticklabels=False, cbar=False, cmap='viridis')
```

Проверим имеются ли значения null, запустив следующий код

```
sns.heatmap(training_set.isnull(), yticklabels=False, cbar=False, cmap='viridis')
```

Осталось большое количество отсутствующих данных в столбце Cabin (Каюты), если бы мы использовали этот столбец для анализа, то тогда можно было бы применить способ что бы предугадать пропущенные значения, но столбец нам не нужен, поэтому мы можем его просто отбросить, при помощи кода представленного ниже

```
training_set.drop('Cabin', axis=1, inplace=True)
```

Проверим нашу таблицу, и увидим что столбец отсутствует

```
training_set.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S

```
sns.heatmap(training_set.isnull(), yticklabels=False, cbar=False, cmap='viridis')
```

Мы видим что осталось одно отсутствующее значение и мы можем легко от него избавиться, код ниже удаляет строки с отсутствующими значениями

```
training_set.dropna(inplace=True)
```

Перезапустим код и видим что мы не имеем отсутствующих значений. Таким образом мы совершили первый шаг очистки данных, мы очистили наши данные от отсутствующих данных, какие то отсутствующие данные мы заполнили средними значениями, а некоторые просто удалили.

Преобразование категориальных признаков

Далее мы будем работать с категориальными характеристиками. Нам необходимо конвертировать категориальные характеристики в численные, так как большинство моделей машинного обучения не может работать с текстовыми данными как входными.

Рассмотрим столбец Sex(пол). Мы видим категории male и female, алгоритм машинного не может принимать строку, т.е. мы должны создать еще один столбец в котором будут нули и единицы, т.е. мы должны закодировать эти категории, чтобы алгоритм машинного обучения мог их понять. Это называется создание фиктивной переменной. И то же самое мы сделаем для столбца Embarked (порт прибытия), так как в этом столбце буквы которые представляют города.

Рассмотрим кодирование категорий целочисленными значениями (label encoding). В этом случае уникальные значения категориального признака кодируются целыми числами. Кодирование категорий целочисленными значениями - LabelEncoder, предполагается что значения категорий заменяются целыми числами в случайном порядке

```
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
```

```
le = LabelEncoder()
cat_enc_le = le.fit_transform(training_set['Sex'])
```

Выведем уникальные значения данного признака:

```
training_set['Sex'].unique()
```

```
array(['male', 'female'], dtype=object)
```

Теперь посмотрим уникальные значения после преобразования:

```
np.unique(cat_enc_le)
```

```
array([0, 1])
```

С помощью созданного объекта можно выполнить обратное преобразование:

```
le.inverse_transform([0,1])
```

```
array(['female', 'male'], dtype=object)
```

В зависимости от данных это преобразование может создать новую проблему. Мы перевели набор стран в набор чисел. Но это всего лишь категориальные данные, и между числами на самом деле нет никакой связи. Проблема здесь в том, что, поскольку разные числа в одном столбце, модель неправильно подумает, что данные находятся в каком-то особом порядке — 0 < 1 < 2 Хотя это, конечно, совсем не так.

Поэтому LabelEncoder можно применять с осторожностью. По этому принципу можно преобразовывать бинарные переменные (такие как пол) или переменные, измеренные по ординальной шкале. Но ординальные переменные надо преобразовывать не в случайном порядке, а в строго определенном, в естественном порядке.

Для решения проблемы мы используем OneHotEncoder. Этот кодировщик берёт столбец с категориальными данными, который был предварительно закодирован в признак, и создаёт для него несколько новых столбцов. Числа заменяются на единицы и нули, в зависимости от того, какому столбцу какое значение присуще.

```
ohe = OneHotEncoder()
cat_enc_ohe = ohe.fit_transform(training_set[['Embarked']])  # Вызываем метод fit_transform, возвращает разреженную матрицу из
```

Выведем изначальную форму датасета:

```
training_set.shape
```

```
(712, 11)
```

Теперь форму преобразованного вектора (изначально это был всего лишь один столбец):

```
cat_enc_ohe.shape
```

```
(712, 3)
```

То же самое преобразование можно сделать при помощи встроенного в pandas метода get_dummies().

Вызовем метод get_dummies передаем столбец Sex, данный метод конвертирует категориальные переменные в фиктивные переменные, так же они известные как переменные-индикаторы

```
pd.get_dummies(training_set['Sex'])
```

	female	male
0	0	1
1	1	0
2	1	0
3	1	0
4	0	1
...	...	...
885	1	0
886	0	1
887	1	0
889	0	1
890	0	1

Получаем такой результат, т.е. у нас два столбца и 0 и 1 в качестве значений, можно расценивать как булевы значения, 0 это False, 1 это True. Первая строка говорит нам о том что male = 1 означает что человек был мужского пола.

Но мы получаем проблему: один столбец идеально предсказывает второй столбец, т.е. если мы зададим такие входные данные в алгоритм машинного обучения, то он поймет что в случае если в одном столбце ноль в другом обязательно будет 1. Эта проблема называется мультиколлинеарностью. Это запутывает алгоритм. Решением будет удаление одного столбца.

```
pd.get_dummies(training_set['Sex'], drop_first=True)
```

	male
0	1
1	0
2	0
3	0
4	1
...	...
885	0
886	1
887	0
889	1
890	1

Присвоим это значение переменной sex

```
sex = pd.get_dummies(training_set['Sex'], drop_first=True)
```

Сделаем тоже самое для столбца Embarked

```
embark = pd.get_dummies(training_set['Embarked'], drop_first=True)
```

У нас было три значения S,C,Q по названиям портов отправки, мы отбросили C у нас осталось два столбца, но они не являются предсказателями друг для друга:

```
embark.head()
```

	Q	S
0	0	1
1	0	0
2	0	1
3	0	1
4	0	1

Добавим вновь созданные столбцы sex, embark в наше множество данных при помощи метода pd.concat мы добавляем список указываем training_set и добавляем столбцы sex, embark и указываем еще один параметр axis=1 что бы указать что это будут столбцы.

```
training_set = pd.concat([training_set, sex, embark], axis=1)
```

Мы видим что у нас остались старые столбцы но так же добавились и новые:

```
training_set.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked	male
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S	1
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S	0
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S	1

Удаляем ненужные столбцы

```
training_set.drop(['Sex', 'Embarked', 'Name', 'Ticket', 'PassengerId'], axis=1, inplace=True)
```

```
training_set.head()
```

	Survived	Pclass	Age	SibSp	Parch	Fare	male	Q	S
0	0	3	22.0	1	0	7.2500	1	0	1
1	1	1	38.0	1	0	71.2833	0	0	0
2	1	3	26.0	0	0	7.9250	0	0	1
3	1	1	35.0	1	0	53.1000	0	0	1
4	0	3	35.0	0	0	8.0500	1	0	1

Таким образом мы получили итоговый набор данных, подготовленный к машинному обучению. Это набор данных удовлетворяет всем условиям чистых данных, и при этом содержит всю информацию из первоначального датасета.

Контрольные вопросы

1. Какие основные виды визуализации вы знаете? Какие у них области применения?
2. Какие типы визуализации больше всего подходят для анализа совместного распределения двух непрерывных переменных?
3. Какие типы визуализации больше всего подходят для анализа совместного распределения двух дискретных переменных?
4. Как лучше всего построить совместное распределение дискретной и непрерывной переменной?
5. Как лучше всего построить совместное распределение двух непрерывных и одной дискретной переменной?
6. Как лучше всего построить совместное распределение двух дискретных и одной непрерывной переменной?

Задания для самостоятельного выполнения

1. Постройте по получившемуся набору данных простую модель машинного обучения и оцените ее эффективность.
2. Ответьте на следующие вопросы при помощи визуализации и численных данных по исходному набору данных:
3. Какова доля выживших после крушения пассажиров? Какова доля мужчин и женщин среди выживших?
4. Сколько пассажиров ехало в каждом классе? Кого было больше в самом многолюдном классе — мужчин или женщин?
5. Все ли признаки несут в себе полезную информацию? Почему? Избавьтесь от ненужных столбцов.
6. Посчитайте, насколько сильно коррелируют друг с другом цена за билет и возраст пассажиров. Также проверьте наличие этой зависимости визуально (в этом вам поможет построение диаграммы рассеяния).
7. Правда ли, что чаще выживали пассажиры с более дорогими билетами? А есть ли зависимость выживаемости от класса?
8. Какова связь между стоимостью билета и портом отправления? Выведите минимальную, среднюю и максимальную сумму, которую заплатили пассажиры за проезд. Проделайте то же самое только для тех пассажиров, которые сели на корабль в Саутгемптоне.
9. Выведите гистограммы, показывающие распределения стоимостей билетов в зависимости от места посадки.
10. Оцените репрезентативность представленной выборки. Сколько всего было пассажиров Титаника? Сколько из них выжило? Какую долю составляет представленный набор данных от всей генеральной совокупности?
11. Разделите выборку на тестовую и обучающую части при помощи train_test_split(). Изобразите на графиках распределение некоторых атрибутов и целевой переменной. Насколько однородно получившееся разбиение?
12. Сбалансируйте классы в исходном датасете двумя способами:
13. Удалите лишние объекты мажоритарного класса (выбранные случайно)
14. Добавьте в выборку дубликаты миноритарного класса.
15. Проведите исследование эффективности простой модели классификации до и после данных преобразований.
16. Постройте корреляционную матрицу признаков после преобразования данных. Сделайте вывод о наличии либо отсутствии мультиколлинеарности признаков.
17. Проведите группировку данных по значению возраста. Введите новый признак "возрастная категория", значениями которой будут "ребенок", "взрослый", "старик". Проведите анализ эффективности данного признака.

Дополнительные задания

1. Выдвиньте и проверьте статистические гипотезы о виде распределения атрибутов исходного набора данных.
2. Проведите автоматизированный отбор признаков тремя методами:
3. Поочередное исключение признаков из модели. Начните с модели, включающей все признаки. Найдите признак, исключение которого приводит к наибольшему повышению эффективности модели. Затем по такому же принципу исключите второй признак, и так до тех пор, пока исключение признаков может повысить эффективность.
4. Поочередное включение признаков. Начните с парной модели с наибольшей эффективностью. Найдите второй признак, включение которого в модель дает наибольший прирост эффективности. Продолжайте добавлять признаки в модель по одному пока это приводит к росту эффективности.
5. Постройте вектор важности признаков. Опираясь на него включайте признаки в модель по одному в порядке уменьшения относительной важности. Найдите набор признаков, который дает наибольшую эффективность.